PROJECT GUIDE · ARCHITECTURE · CODE · INTERVIEW PREP

F1 Strategy Lab

An AI-powered Formula 1 race-strategy analyser that runs entirely on a laptop at zero cost. The backend calculates every statistic from real lap data, and a local LLM only explains those calculated facts.

Java 21+ | Spring Boot 4.1 | Spring Data JPA / Hibernate 7 | H2 (file) | Spring AI 2.0 | Ollama · Llama 3.2 | Angular 22 | TypeScript | Apache ECharts | Jolpica F1 API | JUnit 5 · Mockito · Vitest

Contents

1. Project overview
2. Architecture & system flow
3. Request flows step by step
4. Project structure
5. Backend code walkthrough
6. The analytics engine (algorithms)
7. GenAI layer: grounding the LLM
8. Frontend code walkthrough
9. Database design
10. Testing strategy
11. Design decisions & trade-offs
12. Limitations & future work
13. Interview questions & answers
14. How to run · quick reference
15. Setting up on another machine

1. Project overview

The problem

Race fans and analysts want to understand *why* things happened in a race. Why did a driver drop places after a pit stop? Who was more consistent? Would an earlier stop have helped? General-purpose chatbots answer these questions confidently but make up statistics. This project gives answers that are **grounded in real data**.

What the application does

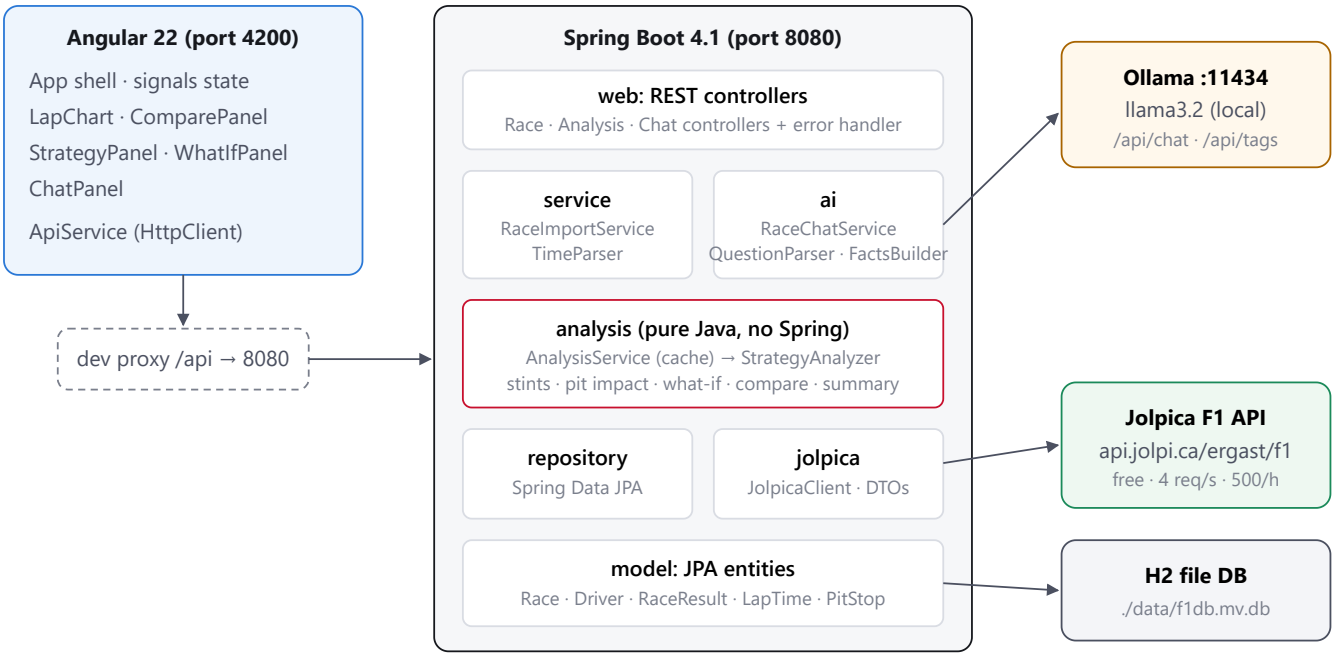
- **Pick a race:** choose a season and race. The first time a race is opened, it is downloaded from the free Jolpica F1 API and stored in a local H2 database. After that it works offline.
- **Dashboard:** race summary tiles, lap-time and running-position charts with pit stops and Safety Car laps marked, and the classification with every driver's strategy.
- **Compare drivers** (up to 3): clean-lap pace, consistency (std dev, coefficient of variation), head-to-head lap counts, and the race-time gap lap by lap.
- **Pit-stop analysis:** time lost, positions lost, cars within the "pit-loss window", and for each rival that got ahead, *why* (undercut, overcut, or "hadn't pitted yet").
- **What-if:** "What if the driver had pitted 5 laps earlier?" This uses a tyre-degradation model fitted to the real lap data.
- **AI chat:** natural-language questions answered by a local LLM, which is only given facts the backend calculated. Each answer shows "Data used".

Hard constraint: ₹0 total cost

Need	Typical paid choice	Choice here (free)
LLM	OpenAI / Claude API tokens	Ollama + Llama 3.2 (3B) running locally on the CPU
Database	Hosted Postgres / RDS	H2 embedded, file-based (`./data/f1db.mv.db`)
F1 data	Commercial sports-data feeds	Jolpica F1 API (Ergast-compatible, no key)
Hosting	Cloud VM / PaaS	localhost (8080 backend, 4200 frontend)

Main idea in one line: *The LLM is a narrator, not a calculator.* Java code computes every number; the model only turns those facts into plain English.

2. Architecture & system flow



Layers and responsibilities

Layer / package	Responsibility
`web`	REST endpoints, request validation, mapping exceptions to HTTP `ProblemDetail`, CORS.
`service`	Importing races from Jolpica into H2 (one-time per race), and time-string parsing.
`jolpica`	HTTP client for the external API: paging, throttling, DTO records.
`model` / `repository`	JPA entities and Spring Data repositories, with flat "row" projections for fast reads.
`analysis`	All the maths. `StrategyAnalyzer` is plain Java over an immutable `RaceData` snapshot, which makes it easy to unit-test.
`ai`	Question parsing, building the fact sheet, and calling Ollama through Spring AI `ChatClient` with a fallback.

Why this split?

- **Pure core, thin shell:** the analysis knows nothing about Spring, JPA or HTTP. It takes `RaceData` in and returns records. You can test it with hand-built data and exact expected numbers.
- **Network only at import time:** the dashboard and chat never call Jolpica, so they are fast and work offline.
- **The AI is replaceable:** the LLM sits behind Spring AI's `ChatClient`. Moving to another local model is a single property change.

3. Request flows step by step

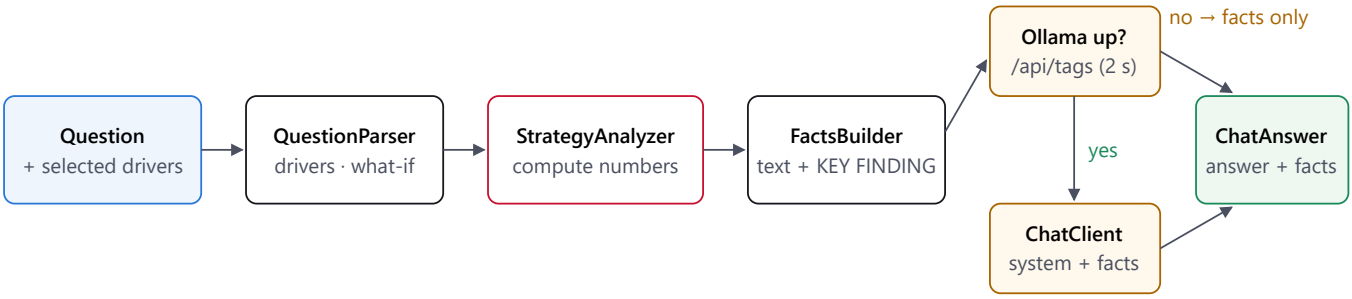
Flow A – Opening a race for the first time (import)

1. The user picks a season. Angular calls `GET /api/seasons/2024/races`. `RaceImportService.schedule()` returns stored races, or fetches the calendar from Jolpica once and saves `Race` rows with `imported=false`.
2. The user picks a race. Angular calls `POST /api/seasons/2024/races/1/import`.
3. If the race is already imported, it is returned immediately (no network). Otherwise `JolpicaClient` fetches `results.json`, then **pages** `laps.json` (100 rows per page, ~12 pages) and `pitstops.json`, sleeping 350 ms between calls to respect the rate limit.
4. Lap pages can split a lap across two pages, so timings are **merged by lap number**.
5. In one `@Transactional` method, it upserts drivers, saves results, bulk-inserts ~1,100 `LapTime` rows (JDBC batch size 500) and the pit stops, then sets `imported=true`.
6. The controller evicts any cached analyser for that race, and Angular then loads the summary, drivers and laps in parallel with `forkJoin`.

Flow B – Dashboard analytics

1. A request such as `GET /api/races/1/drivers/leclerc/pit-impact` reaches `AnalysisController`.
2. `AnalysisService.analyzer(raceId)` returns a cached `StrategyAnalyzer` or builds one. Building it loads `RaceData` using JPQL *constructor projections* (`LapRow`, `PitRow`), so no lazy-loading and no N+1 queries.
3. The analyser's constructor builds indexes once: laps by driver (`TreeMap`), stops by driver, cumulative race time per driver per lap, and the set of suspected SC/VSC laps.
4. The requested method (`pitImpact`, `stints`, `whatIf`, `compare`, `summary`) computes the result and returns immutable records, which Jackson serialises to JSON.
5. Errors become HTTP status codes: `NoSuchElementException` → 404, `IllegalArgumentException` / `IllegalStateException` → 400 (for example "stop 3 does not exist"), Jolpica failures → 502.

Flow C – Asking the AI a question



1. `POST /api/ai/chat` with `{raceId, question, driverIds}`. The body is validated with `@Valid` (`@NotBlank`, `@Size(max=1000)`).
2. `QuestionParser` finds the drivers mentioned (accent-insensitive surnames like "Perez"/"Pérez", and 3-letter codes only when written in capitals) and what-if patterns like "five laps earlier" or "3 laps later". It also picks

the stop number ("second stop").

3. `RaceChatService.buildFacts` always adds the race summary. For each driver (max 3) it adds strategy and pit impact; for two drivers, a comparison; for a what-if, the model result or a note explaining why it couldn't be computed.
4. `OllamaStatusService` checks `/api/tags` (2 s timeout). If Ollama is down or the model isn't pulled, it immediately returns the facts with an explanatory message, so the request never hangs.
5. Otherwise `ChatClient` sends a strict system prompt plus `FACTS:… QUESTION:…`. The temperature is 0.2 to keep answers factual.
6. The response contains `answer`, the exact `facts` text, `driversUsed`, `llmUsed` and `model`. The UI shows the facts under "Data used".

4. Project structure

```
AI/                          (Spring Boot
backend)
├─ pom.xml                   Boot 4.1.1, Spring
AI 2.0.1
├─ data/f1db.mv.db           H2 database
(runtime)
└─ src/main/java/com/f1/AI/
   ├─ AiApplication.java
@SpringBootApplication
   │                         @EntityScan
   ├─ model/                 JPA entities
   │   Race, Driver, RaceResult,
   │   LapTime, PitStop
   ├─ repository/            Spring Data +
projections
   ├─ jolpica/               JolpicaClient,
JolpicaDtos
   ├─ service/               RaceImportService,
   │                         TimeParser
   ├─ analysis/              RaceData,
AnalysisModels,
   │                         StrategyAnalyzer,
   │                         AnalysisService
   ├─ ai/                    QuestionParser,
   │                         FactsBuilder,
   │                         RaceChatService,
   │                         OllamaStatusService
   └─ web/                   Controllers,
WebConfig,
                             ApiExceptionHandler
   resources/application.properties
   test/…                    22 JUnit tests
```

```
f1-ui/                       (Angular 22
frontend)
├─ proxy.conf.json           /api →
localhost:8080
├─ angular.json
└─ src/
   ├─ styles.css             design tokens,
light/dark
   ├─ main.ts
   └─ app/
      ├─ app.ts / app.html   shell, race picker,
      │                      driver selection,
layout
      ├─ app.config.ts       HttpClient, ECharts
      ├─ api.service.ts      typed REST client
      ├─ models.ts           TS interfaces =
DTOs
      ├─ format.ts           lap-time formatting
      ├─ chart-theme.ts      CSS tokens →
ECharts
      ├─ lap-chart.ts        lap time / position
      ├─ compare-panel.ts    head-to-head + gap
      ├─ strategy-panel.ts   stints + pit stops
      ├─ what-if-panel.ts    pit-stop simulation
      ├─ chat-panel.ts       AI chat
      └─ app.spec.ts         Vitest tests
```

REST API

Method	Path	Purpose
GET	`/api/seasons/{season}/races`	Season calendar (cached in H2)
POST	`/api/seasons/{s}/races/{r}/import?force=false`	Download one race into H2
GET	`/api/races`	Imported races
GET	`/api/races/{id}/summary`	Winner, podium, strategies, gainers/losers, SC laps
GET	`/api/races/{id}/laps`	Chart data: lap times, positions, pit laps
GET	`/api/races/{id}/drivers/{d}/strategy`	Stints, pace, consistency, degradation
GET	`/api/races/{id}/drivers/{d}/pit-impact`	Per-stop time and positions lost, and why
GET	`/api/races/{id}/drivers/{d}/what-if?stop=1&lapsEarlier=5`	Moved-stop estimate (negative = later)
GET	`/api/races/{id}/compare?a=..&b=..`	Head-to-head and gap by lap
POST	`/api/ai/chat`	Grounded AI answer + facts used
GET	`/api/ai/status`	Ollama running? Model pulled?

5. Backend code walkthrough

5.1 Entities (model)

Five entities. `Race` has a unique constraint on `(season, race_round)`. `Driver` uses Jolpica's natural key (`"max_verstappen"`) as its `@Id`, so re-importing upserts instead of duplicating. Associations are `LAZY` by default to avoid loading whole graphs.

```
@Entity
@Table(uniqueConstraints = @UniqueConstraint(columnNames = {"season", "race_round"}))
public class Race {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY) private Long id;
    private int season;
    @Column(name = "race_round") private int round;   // "ROUND" is reserved in H2
    private boolean imported;
    ...
}

@Entity @Table(indexes = @Index(columnList = "race_id, driver_id"))
public class LapTime {
    @ManyToOne(fetch = FetchType.LAZY, optional = false) private Race race;
    @ManyToOne(fetch = FetchType.LAZY, optional = false) private Driver driver;
    @Column(name = "lap_number")   private int lap;
    @Column(name = "lap_position") private int position;
    private long timeMs;
}
```

Bug found during development: `round` and `position` clash with SQL/H2 reserved words, so the columns were renamed. The unique constraint must name the *column* (`race_round`), not the field. A mismatch made Hibernate's DDL fail silently and the `race` table was never created.

5.2 Repositories with constructor projections

Analysis needs thousands of rows but only four columns. Loading entities would mean lazy proxies and possible N+1 queries. A JPQL `select new …` projection into a record returns exactly what's needed in one query:

```
public interface LapTimeRepository extends JpaRepository<LapTime, Long> {
    record LapRow(String driverId, int lap, int position, long timeMs) {}

    @Query("select new com.f1.AI.repository.LapTimeRepository$LapRow(" +
           "l.driver.id, l.lap, l.position, l.timeMs) " +
           "from LapTime l where l.race.id = :raceId order by l.lap, l.position")
    List<LapRow> findRows(Long raceId);

    @Modifying
    @Query("delete from LapTime l where l.race.id = :raceId")
    void deleteByRaceId(Long raceId);       // one bulk DELETE, not N selects + deletes
}
```

5.3 JolpicaClient – paging, merging, throttling

```
public List<LapDto> laps(int season, int round) {
    Map<String, List<TimingDto>> byLap = new LinkedHashMap<>();
    int offset = 0, total;
    do {
        Envelope page = get("/{season}/{round}/laps.json?limit={limit}&offset={offset}",
                            season, round, PAGE_SIZE, offset);
        total = parseInt(page.mrData().total());
        for (RaceDto race : races(page))
            for (LapDto lap : race.laps())
                byLap.computeIfAbsent(lap.number(), k -> new ArrayList<>()).addAll(lap.timings());
        offset += PAGE_SIZE;
    } while (offset < total);
    ...
}
```

- **Paging:** Jolpica pages by *timing row* (max 100), not by lap, so a lap can be split across pages. Merging by lap number in a `LinkedHashMap` fixes this.
- **Throttling:** `Thread.sleep(350)` before each call keeps the client under the public limit of 4 req/s and 500 req/h. One race is about 16 requests.
- **Timeouts:** 10 s connect, 30 s read on `SimpleClientHttpRequestFactory`, so a slow API can't block threads forever.
- **DTOs:** Java records with `@JsonProperty("MRData")` etc. Every number arrives as a string and is parsed defensively (`positionText` can be "R", "D" or "W").

5.4 RaceImportService

```
@Transactional
public Race importRace(int season, int round, boolean force) {
    Race race = races.findBySeasonAndRound(season, round).orElseGet(() -> new Race(season, round));
    if (race.isImported() && !force) return race;          // idempotent, no network
    RaceDto resultDto = jolpica.results(season, round)
        .orElseThrow(() -> new NoSuchElementException("No results ... (race not run yet?)"));
    List<LapDto> lapDtos = jolpica.laps(season, round);
    List<PitStopDto> pitDtos = jolpica.pitStops(season, round);
    ... delete old rows (re-import), save drivers/results, saveAll(laps), saveAll(stops)
    race.setImported(true);
    return races.save(race);
}
```

Everything happens in one transaction: if anything fails, nothing half-imported is left behind. The fetches happen *before* any writes, so a network failure doesn't touch the database. `hibernate.jdbc.batch_size=500` and `order_inserts=true` turn ~1,100 inserts into a few JDBC batches.

5.5 AnalysisService – cache and load

```
private final Map<Long, StrategyAnalyzer> cache = new ConcurrentHashMap<>();

@Transactional(readOnly = true)
public StrategyAnalyzer analyzer(long raceId) {
    StrategyAnalyzer cached = cache.get(raceId);
    if (cached != null) return cached;
    StrategyAnalyzer analyzer = new StrategyAnalyzer(load(raceId));
    cache.put(raceId, analyzer);
    return analyzer;
}
public void evict(long raceId) { cache.remove(raceId); }   // called after (re)import
```

Race data never changes after import, so caching the analyser (with its precomputed indexes) makes every later request purely in-memory. `ConcurrentHashMap` makes it safe to use from Tomcat's request threads. Two threads might occasionally build the same analyser at once, which is harmless because they produce the same result.

5.6 Controllers and error handling

```
@RestControllerAdvice
public class ApiExceptionHandler {
    @ExceptionHandler(NoSuchElementException.class)
    public ProblemDetail notFound(NoSuchElementException e) {
        return ProblemDetail.forStatusAndDetail(HttpStatus.NOT_FOUND, e.getMessage());
    }
    @ExceptionHandler({IllegalArgumentException.class, IllegalStateException.class})
    public ProblemDetail badRequest(RuntimeException e) { ... 400 ... }
    @ExceptionHandler(RestClientException.class)
    public ProblemDetail upstream(RestClientException e) { ... 502 Bad Gateway ... }
}
```

`ProblemDetail` is the RFC 9457 standard error body. The Angular `errorMessage()` helper reads `err.error.detail`, so users see messages like "VER made 2 stop(s); stop 3 does not exist".

5.7 Configuration highlights

```
spring.datasource.url=jdbc:h2:file:./data/f1db;AUTO_SERVER=TRUE
spring.jpa.hibernate.ddl-auto=update
spring.jpa.open-in-view=false                # no lazy loading from controllers/views
spring.h2.console.enabled=true               # http://localhost:8080/h2-console
spring.ai.ollama.chat.model=llama3.2
spring.ai.ollama.chat.temperature=0.2
spring.ai.ollama.chat.num-ctx=4096
spring.ai.retry.max-attempts=2               # fail fast instead of minutes of backoff
f1.jolpica.request-delay-ms=350
```

6. The analytics engine (algorithms)

`StrategyAnalyzer` is constructed once per race and builds these indexes:

- `lapsByDriver: Map<driverId, TreeMap<lap, LapRow>>`: ordered, with O(log n) lookup of any lap.
- `cumulativeByDriver`: running total of lap times, which gives the race time at the end of every lap. Gaps between cars are differences of these totals.
- `neutralisedLaps`: the suspected Safety Car / VSC laps.

6.1 Safety Car / VSC detection

Jolpica has no Safety Car flag, so it is inferred: for every lap (except lap 1) take the **median** lap time across the field. Then take the median of those medians as a "typical lap". A lap whose field median is more than **15%** slower is flagged as neutralised.

Why the median? One car crawling to the pits or crashing shouldn't flag a lap; the whole field has to slow down.

6.2 Clean laps

Pace and consistency statistics only use *representative* laps. A lap is excluded if it is lap 1 (standing start), a pit in-lap or out-lap, a suspected SC/VSC lap, or slower than **107%** of the driver's own median (traffic, mistakes, yellow flags).

6.3 Stints and degradation (linear regression)

Pit-stop laps split the race into stints. For each stint, clean lap times are fitted against **tyre age** (laps since the stint started) using least squares:

```
slope = (n·Σxy − Σx·Σy) / (n·Σx² − (Σx)²)        intercept = (Σy − slope·Σx) / n
// x = tyre age, y = lap time (ms).  slope = ms lost per lap of tyre age ("degradation")
// requires ≥ 3 clean laps and non-zero x variance, otherwise null
```

Each stint also reports its average, median, std dev (sample, n−1) and best lap. Consistency is the **coefficient of variation** = std dev ÷ mean × 100%. It is scale-free, so drivers on different pace can be compared fairly.

6.4 Pit-stop impact – "why did they lose places?"

1. **Positions:** before = position at lap `p−1` (grid if `p=1`), after = end of the out-lap `p+1`, and 3 laps later.
2. **Time lost** = (in-lap + out-lap) − 2 × reference pace, where reference = median clean lap within ±5 laps.
3. **Pit-loss window:** every car behind at lap `p−1` whose gap is less than the time lost. These cars are *expected* to come out ahead.
4. **Rival classification** for every car that got ahead: pitted earlier → *undercut*; same lap → came out ahead; did not pit within ±5 laps → *gained track position*; pitted later → check positions after *their* stop. If our driver was ahead again, the place was *temporary*; otherwise the *overcut worked*.

Real example (Bahrain 2024, Leclerc stop 1, lap 11): 23.644 s lost, P4 → P13, 12 cars inside the pit-loss window, but 9 of the cars that passed hadn't pitted yet and the places were temporary. That explains a big drop that looks alarming but was mostly temporary.

6.5 What-if: moving a pit stop

Inputs: driver, stop number `s`, `k` laps earlier (negative = later). Original pit lap `p`, new pit lap `p' = p − k`.

```
fitA = regression on stint before the stop    (age = lap − stintStart + 1)
fitB = regression on stint after the stop     (age = lap − p)
pitLoss = actual(p) + actual(p+1) − fitA(age at p) − fitB(1)   // carried over from the real stop

for lap in [min(p,p') .. end of next stint]:
    lap < p'   → real time if still before original stop, else fitA(age)   // older tyres
    lap == p'  → (same as above) + pitLoss                                  // new in-lap
    lap > p'   → fitB(lap − p')                                             // fresher tyres
    SC/VSC laps keep their real time in both scenarios

delta = modelled − real          // negative = faster
estimated finish = re-rank using final cumulative race times ± delta
```

Validation rejects impossible inputs with 400 errors: a stop that doesn't exist, a new lap that would erase the previous stint, or one that collides with the next stop or the finish. **Warnings** are returned when the new stint is more than 3 laps longer than anything observed (linear extrapolation), or when the real stop was under a Safety Car (its pit loss was unrealistically cheap).

Assumptions (also returned in the API): linear degradation per stint, same pit loss, no traffic, no rival reaction, unknown compounds, fuel effect only implicit in each stint's slope.

6.6 Compare and summary

- **Head-to-head:** only laps where *both* drivers were clean are compared, giving a mean delta and a count of laps each driver was faster. The gap by lap is the difference of cumulative times.
- **Summary:** winner, podium, fastest lap, most common stop count, biggest gainers/losers (grid → finish; grid 0 = pit-lane start, treated as last), retirements, SC ranges (consecutive laps collapsed: "laps 6-7"), stops under SC, and quickest pit-lane time.

7. GenAI layer: grounding the LLM

Why not just give the model the raw data?

- Raw lap data is ~1,100 rows, too large for a small model's context window and very slow on a CPU.
- LLMs are bad at arithmetic. Asking a 3B model to compute regressions or cumulative gaps leads to made-up numbers.
- The design is a **deterministic retrieval-and-compute** pipeline (a form of RAG where "retrieval" is calculation). Code decides *what* is true, and the model decides *how to say it*.

Why not LLM tool-calling?

Spring AI supports `@Tool` functions, but small local models choose tools unreliably and make up arguments. Parsing the question in code (regex and name matching) is deterministic, testable and fast. Tool-calling would be a natural upgrade with a larger model.

The system prompt

```
You are a Formula 1 race strategy analyst explaining data to a fan.
Rules:
- Use ONLY the FACTS. Every number you write must appear in the FACTS, copied exactly.
- If a line says KEY FINDING, base your answer on it.
- Do not invent causes that are not in the FACTS (no crew errors, damage, team orders or weather).
- Tyre compounds are NOT in the data: never mention soft, medium or hard tyres.
- If the FACTS do not answer the question, say what is missing.
- Only for a what-if question: give the estimated time difference and finish, then the warnings.
Answer in plain English in 3 to 6 short sentences or bullet points. No headings.
```

Prompt-engineering lessons learned (measured on real answers)

Problem seen	Fix	Result
Model reversed meaning ("0.373 s behind SAI" when SAI was behind)	Unambiguous wording: "Cars BEHIND LEC before the stop…"	Correct direction
Model invented "pit crew error"	Explicit "do not invent causes" rule plus a deterministic **KEY FINDING** sentence	Answer matched the finding
What-if headings on non-what-if questions	Made the what-if rule conditional	Plain prose
102 s per answer on a CPU	Dropped the 20-driver classification for driver questions; `num-ctx` 8192 → 4096	27–40 s
Request hung for minutes when Ollama was down	Fast health check plus `spring.ai.retry.max-attempts=2`	~1 s fallback

Calling the model with Spring AI

```
this.chatClient = chatClientBuilder.defaultSystem(SYSTEM_PROMPT).build();

String answer = chatClient.prompt()
        .user(u -> u.text("FACTS:\n{facts}\n\nQUESTION: {question}")
                    .param("facts", facts)
                    .param("question", question))
        .call()
        .content();
```

Template parameters keep user input separate from instructions. `ChatClient.Builder` is auto-configured by `spring-ai-starter-model-ollama` from `spring.ai.ollama.*` properties.

Graceful degradation

The app is fully useful without the LLM: if Ollama is down, the model isn't pulled, or the call throws, `ChatAnswer.llmUsed=false` and the user still gets the computed facts. The UI badge shows "● llama3.2 (local)" or "○ AI offline", along with a hint.

> **Known limitation:** a 3B model can still misread facts. For example, it once called *both* drivers "more consistent" when the KEY FINDING named one. That's why every answer shows "Data used". Switching to `qwen2.5:7b` (one property) improves reliability at roughly twice the latency.

8. Frontend code walkthrough

Angular 22 – modern style

- **Standalone components** (no NgModules) and the new control flow (`@if`, `@for … track`).
- **Signals** for state: `signal()` for writable state, `computed()` for derived values (chart options, the selected pair), and `effect()` to reload when inputs change. `input.required<T>()` provides signal-based inputs.
- **Zoneless** change detection (the Angular 22 default): signals tell Angular exactly what changed.

```
// strategy-panel.ts – reload whenever raceId or driverId changes, cancel stale requests
constructor() {
  effect((onCleanup) => {
    const sub = forkJoin([this.api.strategy(this.raceId(), this.driverId()),
                          this.api.pitImpact(this.raceId(), this.driverId())])
      .subscribe({ next: ([s, p]) => { this.strategy.set(s); this.impact.set(p); },
                   error: (e) => this.error.set(errorMessage(e)) });
    onCleanup(() => sub.unsubscribe());
  });
}
```

Key pieces

`ApiService`	Typed `HttpClient` wrapper; one method per endpoint. `errorMessage()` extracts the `ProblemDetail.detail` or explains "backend not reachable" (status 0).
`models.ts`	TypeScript interfaces mirroring the Java records, giving end-to-end type safety.
`proxy.conf.json`	Dev server forwards `/api` to `localhost:8080`, so the same origin is used and there are no CORS issues in development. A CORS config also exists for direct calls.
`App`	Season/race picker, deep links (`?season=2024&round=1` via `history.replaceState`), driver chips (max 3), and the layout grid with a sticky chat panel.
`LapChart`	Lap-time / position toggle. Pit laps are dashed `markLine`s; SC laps are shaded `markArea`s; end labels name each line directly; optionally hides lap 1, pit and SC laps (as `null` gaps).
`ComparePanel`	Head-to-head table and gap chart with a zero baseline.
`WhatIfPanel`	Template-driven form (`FormsModule`) calling the what-if endpoint and showing the delta, estimated finish, warnings and assumptions.
`ChatPanel`	Message list, suggested questions built from the selected drivers' names, a collapsible "Data used" section, and the Ollama status badge.

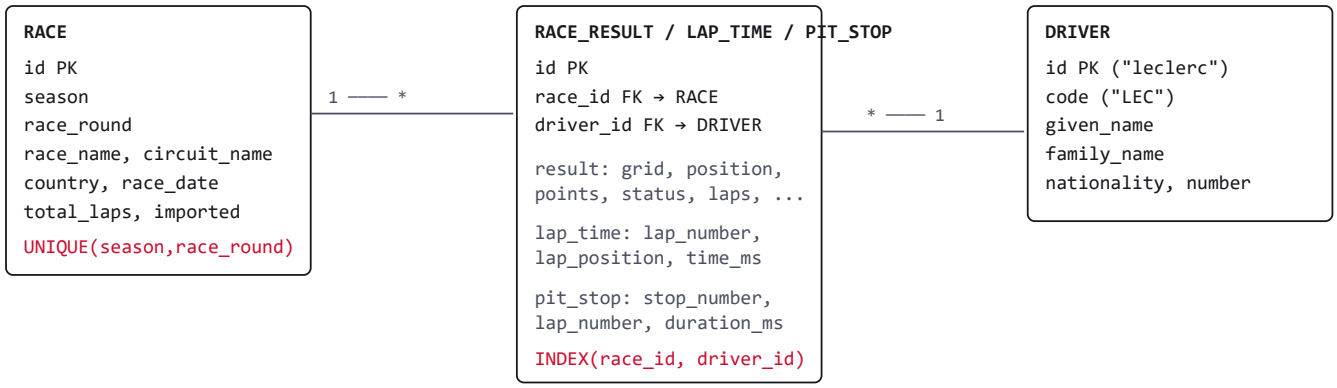
Charts done properly

- **Colours come from CSS tokens** (`--series-1..3`). `ChartTheme` reads them with `getComputedStyle` and re-reads when the OS switches light/dark, so ECharts follows the theme.
- **The colour follows the driver, not their position in the list:** each selected driver keeps their colour slot until deselected, so removing one driver never repaints the others.
- **Max 3 drivers:** only the first three palette slots stay distinguishable under every type of colour blindness. A fourth line would be ambiguous.

- 2 px lines, recessive grid, tooltips with formatted lap times, direct labels, and only one y-axis (no dual-axis charts).

```
// app.ts — stable colour slots
protected toggle(d: DriverInfo): void {
  const current = this.selected();
  if (this.isSelected(d.id)) { this.selected.set(current.filter(s => s.id !== d.id)); return; }
  if (current.length >= MAX_SELECTED) return;
  const used = new Set(current.map(s => s.slot));
  const slot = [0, 1, 2].find(i => !used.has(i))!;
  this.selected.set([...current, { id: d.id, code: d.code, slot }]);
}
```

9. Database design



- **Normalised:** drivers are stored once and shared across races. Results, laps and pit stops are child rows of a race.
- **Composite index** `(race_id, driver_id)` on laps and pit stops matches the main access pattern.
- **Times stored as** `long` **milliseconds:** exact integer maths, no floating-point drift, easy summing.
- **Schema** via `ddl-auto=update`, which is fine for a local single-user tool. Production would use Flyway/Liquibase migrations.
- `AUTO_SERVER=TRUE` lets a second process (the H2 console, an IDE, a second app instance) open the same file safely.
- Tests use an in-memory H2 (`jdbc:h2:mem:f1test`, `create-drop`), so they never touch real data.

10. Testing strategy

Test class	What it proves
`StrategyAnalyzerTest` (9)	A synthetic 20-lap, 3-driver race with exact numbers, so every expected value can be worked out by hand: stints and 0.100 s/lap degradation, pit impact (P1→P3, 20.100 s lost, rival explanations), what-if delta exactly +2.500 s, gap −8.500 s, SC detection, range collapsing, invalid-input errors.
`QuestionParserTest` (4)	Mention order, accent-insensitive names, uppercase-only codes ("per" is not Pérez), what-if parsing in digits and words, earlier vs later, stop ordinals.
`RaceChatServiceTest` (5)	Facts contain the right sections and KEY FINDINGs; classification omitted for driver questions; LLM skipped when Ollama is down (`verify(client, never()).prompt()`); fallback when the call throws. Uses Mockito deep stubs for the fluent `ChatClient` API.
`TimeParserTest` (3)	"1:37.284", "22.345", "1:31:44.742" → ms; blank rejected; formatting.
`AiApplicationTests` (1)	The full Spring context starts (entity scan, repositories, Spring AI auto-config) against in-memory H2. This test caught the entity-scan and DDL bugs.
`app.spec.ts` (Vitest, 2)	App loads the season calendar via `HttpTestingController` ; formatting helpers.

Testing approach: the maths is tested with *hand-calculable synthetic data*, not by snapshotting real-race output. A real-data snapshot only proves the code is unchanged; hand-calculated data proves it is *correct*. Writing the tests even exposed a mistake in my own expectation: a non-stopping third car also got past during the stop.

11. Design decisions & trade-offs

Decision	Why	Trade-off
Compute in Java, LLM only narrates	No made-up stats; testable; works without the LLM	Every new question type needs code
Regex/name parsing instead of tool-calling	Deterministic and reliable with a 3B model	Less flexible phrasing
Import once, then offline	Speed, rate limits, offline use	Data is a snapshot; `force=true` refreshes
H2 file DB	Zero install, zero cost, SQL + JPA like a real DB	Single-node; not for concurrent multi-user production
In-memory analyser cache	Immutable data → perfect cache hits	Memory grows with races viewed (small: ~1k rows each)
Pure `StrategyAnalyzer`	Unit tests without Spring or a DB	Needs a loader layer (`AnalysisService`)
Linear degradation model	Simple, explainable, robust with few laps	Real tyre wear is non-linear (cliffs); fuel effect blended in
Median-based SC detection	Robust to single-car outliers	Inferred, not official; threshold is a heuristic
Fail-fast health check	Instant fallback instead of a hung request	One extra 2 s-timeout call per question
Signals + standalone Angular	Less boilerplate, fine-grained updates	Requires newer Angular knowledge

12. Limitations & future work

Current limitations

- No tyre-compound, telemetry, sector or weather data (Jolpica doesn't provide it).
- SC/VSC laps are inferred, not official race-control messages.
- The what-if model ignores traffic, dirty air and rival reactions.
- A 3B LLM can occasionally misread facts, which is why every answer has "Data used" for checking.
- Answers take 25–40 s on CPU and are not streamed.
- Single-user, local; no authentication.

Future improvements

- **OpenF1 API** (free) for tyre compounds, stints and car data for 2023+ races.
- **Streaming responses** (Spring AI `.stream()` + Server-Sent Events) so text appears token by token.
- **Tool-calling** with a larger model (`qwen2.5:7b`) so the LLM can request extra calculations.
- **Answer validation:** check that every number in the answer appears in the facts; regenerate or flag if not.
- Non-linear degradation (piecewise or quadratic fit), fuel correction (~0.03 s/lap), traffic modelling.
- Flyway migrations, Postgres in Docker, an async import job with progress updates.
- Docker Compose for one-command start; Playwright end-to-end tests.

13. Interview questions & answers

Labels: Basic Intermediate Advanced. Answer in your own words; these are model answers based on this project.

13.1 Project & architecture

Q1. Give a 60-second overview of the project. Basic

It's a full-stack F1 strategy analyser. Angular shows charts and a chat; Spring Boot imports real race data from the free Jolpica API into H2 and computes stints, pit-stop impact, comparisons and what-if simulations. A local LLM through Ollama and Spring AI explains those results in plain English. The key design choice is that the LLM never computes anything; it only narrates facts the backend calculated, so answers stay grounded. It all runs locally at zero cost.

Q2. Why did you choose this architecture? Intermediate

A layered Spring app (web → service → analysis → repository) with a pure-Java analysis core. The core doesn't depend on Spring or JPA, so it's fast and fully unit-testable with synthetic data. External calls happen only at import time, so the dashboard and chat are offline and fast. The LLM is behind Spring AI's ChatClient, so it's replaceable.

Q3. What was the hardest part? Intermediate

Making a small local model give correct explanations. The first answers reversed facts and invented causes like "pit crew error". I fixed it on the data side rather than only in the prompt: unambiguous wording, a deterministic KEY FINDING sentence, shorter context and stricter rules. Latency also dropped from 102 s to about 30 s.

Q4. How does a request flow end to end when a user asks a question? Intermediate

Angular POSTs to /api/ai/chat. The controller validates the request. QuestionParser finds drivers and what-if intent. StrategyAnalyzer computes strategy, pit impact, comparison and what-if. FactsBuilder renders compact text. OllamaStatusService checks the model is available. ChatClient sends the system prompt plus the facts. The answer and the facts go back, and the UI shows the facts under "Data used".

Q5. How did you meet the zero-cost requirement? Basic

Ollama with Llama 3.2 runs locally instead of paid LLM APIs, the H2 file database replaces a hosted one, the free keyless Jolpica API provides the data, and everything runs on localhost. All libraries are open source.

Q6. How would you deploy this for many users? Advanced

Swap H2 for Postgres with Flyway migrations. Move the analyser cache to Caffeine with size limits, or Redis if there are several instances. Run imports as background jobs, ideally pre-loading whole seasons. Put the LLM on a GPU server (Ollama or vLLM) behind a queue with rate limits, and stream responses over SSE. Containerise with Docker, add authentication, observability (Micrometer/Prometheus) and caching of common answers.

13.2 Spring Boot & REST

Q7. What does @SpringBootApplication do? Basic

It combines @Configuration, @EnableAutoConfiguration and @ComponentScan. Auto-configuration created the DataSource from properties, JPA, Tomcat and Spring AI's ChatClient.Builder. Component scan finds controllers and services under com.f1.AI.

Q8. Why did you need @EntityScan? Intermediate

The context test failed with "Not a managed type: Race". Pointing @EntityScan (in Spring Boot 4 it lives in org.springframework.boot.persistence.autoconfigure) explicitly at com.f1.AI.model registered the entities. Lesson: a context-load test catches wiring problems that unit tests can't.

Q9. How is dependency injection used? Basic

Constructor injection everywhere, with final fields and no @Autowired on fields. This makes dependencies explicit and immutable, and lets tests create services with mocks directly, as in `new RaceChatService(analysis, ollama, builder, "llama3.2")`.

Q10. How do you handle errors in the API? Intermediate

A @RestControllerAdvice maps exceptions to RFC 9457 ProblemDetail responses: NoSuchElementException → 404, IllegalArgument/IllegalState → 400 with a readable message, RestClientException → 502 for upstream failures. The frontend shows the detail field.

Q11. How do you validate input? Basic

Bean Validation on the chat request record (@NotNull raceId, @NotBlank and @Size(max=1000) question) with @Valid on @RequestBody. Business validation (does the stop exist, is the new pit lap legal) happens in the analyser and is reported as a 400.

Q12. How did you configure CORS? Intermediate

A WebMvcConfigurer allows GET/POST on /api/** from a configurable origin (localhost:4200). In development the Angular proxy makes requests same-origin anyway, so CORS is a fallback for calling the API directly.

Q13. Why RestClient instead of RestTemplate or WebClient? Intermediate

RestClient is Spring's modern synchronous fluent client. It suits blocking MVC code without pulling in reactive types. I configured explicit connect/read timeouts on the request factory so slow upstreams can't hang threads.

Q14. What is spring.jpa.open-in-view=false and why set it? Advanced

Open-in-view keeps the Hibernate session open during view rendering, which hides lazy-loading N+1 problems and holds connections longer. Disabling it forces all data access to happen inside service transactions. Here, controllers only return records built from projections, so nothing lazy leaks out.

13.3 JPA, Hibernate & H2

Q15. What is the N+1 problem and how did you avoid it? Intermediate

Loading N parent entities and then lazily loading each child runs N+1 queries. For analysis I used JPQL constructor projections (`select new LapRow(l.driver.id, l.lap, …)`) that fetch flat rows in one query, with no entity graphs and no proxies.

Q16. LAZY vs EAGER: what did you choose? Basic

LAZY for race and driver on laps and pit stops, which have many rows. RaceResult → Driver is EAGER because results are few (~20) and are always shown with the driver's name.

Q17. How do you make the import efficient and safe? Intermediate

All remote fetches happen first; then one @Transactional method writes everything, so a failure rolls back completely. Bulk JPQL deletes clear old rows on re-import. JDBC batching (batch_size=500, order_inserts) cuts ~1,100 inserts down to a few round trips.

Q18. Why a natural key for Driver but a surrogate key for Race? Intermediate

Jolpica's driverId is stable and globally unique, so using it as the @Id makes saving idempotent (re-import upserts). Races get a surrogate IDENTITY id for compact foreign keys, plus a unique constraint on (season, race_round) as their business key.

Q19. What went wrong with the column named "round"? Advanced

ROUND is reserved in H2. I renamed the column to race_round but forgot that @UniqueConstraint uses column names, not field names. Hibernate's DDL failed and logged only a warning, so the table was never created. I found it by reading the DDL warnings in the context test log. Lesson: treat schema warnings as errors.

Q20. H2 vs a production database? Basic

H2 is embedded and needs no installation, which is ideal for a free local tool and for tests (in-memory). For production I'd use Postgres with Flyway migrations instead of ddl-auto=update, connection pooling tuned for the load, and backups.

Q21. What does AUTO_SERVER=TRUE do? Intermediate

The first process to open the H2 file also starts a small TCP server, so later processes (the H2 console, IDE tools, a second app instance) connect through it instead of failing on the file lock.

13.4 GenAI, LLMs & Spring AI

Q22. How do you prevent hallucinations? Intermediate

Four layers. (1) The model gets only computed facts, never raw data or the job of calculating. (2) The system prompt requires every number to come from the facts and bans invented causes and tyre compounds. (3) Low temperature (0.2). (4) Transparency: the UI shows the facts used. A next step would be automatically checking that every number in the answer appears in the facts.

Q23. Is this RAG? Intermediate

It's RAG-like, with deterministic retrieval instead of vector search: the "retrieved context" is calculated facts chosen by intent parsing. Vector search suits unstructured text. For structured numeric data, calculating exactly what's needed is more accurate and cheaper.

Q24. Why Ollama and Llama 3.2 3B? Basic

Ollama is free, easy to install on Windows and exposes a local HTTP API that Spring AI supports. A 3B model (~2 GB) runs on a laptop CPU. With 32 GB RAM, qwen2.5:7b is an option for better reasoning at about twice the latency, and it's a single property change.

Q25. What is Spring AI's ChatClient and why use it? Intermediate

It's a fluent, provider-neutral API (prompt → system/user → call → content). Auto-configuration builds it from properties. It supports templates, advisors, streaming, structured output and tool-calling, so moving to another provider wouldn't change the business code.

Q26. Why not use function/tool-calling? Advanced

Small local models choose tools and arguments unreliably. Deterministic parsing in Java is testable and predictable. With a larger model, I'd expose analyser methods as @Tool functions so the model can ask for follow-up calculations itself.

Q27. What are temperature and num-ctx? Basic

Temperature controls randomness: low values make answers focused and repeatable, which suits factual explanations. num-ctx is the context window in tokens. A smaller window uses less memory and is faster on a CPU, so I reduced it to 4096 once the facts were trimmed.

Q28. How do you handle the LLM being unavailable? Intermediate

A 2 s health check on /api/tags runs before calling the model. If Ollama is down or the model isn't pulled, the user gets the computed facts immediately. I also cut Spring AI's retry to 2 attempts, because its default exponential backoff made requests hang for minutes. Any exception during the call also falls back to the facts.

Q29. How did you improve answer quality: prompt or data? Advanced

Mostly the data. Small models do better with short, unambiguous, labelled lines. I put the conclusion into a deterministic KEY FINDING sentence, made directions explicit ("cars BEHIND X"), and removed irrelevant context. The prompt tells the model to base its answer on the KEY FINDING. I measured before and after on the same questions.

Q30. How would you evaluate the AI systematically? Advanced

Build a golden set of questions per race with expected facts. Automatically check that every number in the answer appears in the facts, that the drivers mentioned are correct, and that no banned terms (compounds, invented causes) appear. Track latency and pass rate per model and prompt version, and use the set as regression tests when changing prompts or models.

Q31. How do you protect against prompt injection? Advanced

The question goes in as a template parameter, separate from the system prompt. The model has no tools and no access to anything except the facts, so the worst case is a strange answer, never an action. Input length is capped at 1,000 characters. In production I'd also filter the output and log answers for review.

13.5 Algorithms & domain logic

Q32. How do you detect Safety Car laps without official data? Intermediate

Take the field-median time for each lap, then the median of those as a typical lap. A lap whose median is more than 15% slower is flagged. Medians make it robust to one slow car, and SC laps are then excluded from pace statistics and kept as real times in the what-if.

Q33. What is a "clean lap" and why does it matter? Basic

A representative green-flag lap: not lap 1, not an in/out-lap, not SC/VSC, and within 107% of the driver's median. Without filtering, pit laps (20+ s slower) would distort averages, standard deviation and degradation slopes.

Q34. How do you measure tyre degradation? Intermediate

A least-squares linear regression of clean lap time against tyre age within each stint. The slope is the time lost per lap of age. At least 3 points are required; with fewer, the slope is null, or the what-if falls back to flat pace and adds a warning.

Q35. Explain the what-if model. Advanced

Fit the stints before and after the stop. Take the real pit loss as (in-lap + out-lap) minus predicted pace. For the new pit lap, rebuild the affected laps: older tyres before the new stop use fitA, the new in-lap adds the pit loss, and later laps use fitB at the new tyre age. SC laps keep their real times. The delta is modelled minus real time, and the finish is re-ranked using final cumulative race times. It validates inputs and returns warnings and assumptions.

Q36. How do you explain why a driver lost positions after a stop? Advanced

Compute the time lost against nearby clean pace and list cars behind by less than that amount (they're expected to pass). Then classify each car that got ahead: an undercut (pitted earlier), staying out, or not stopped yet. For the last case, check positions after the rival's own stop to tell a temporary place from a successful overcut.

Q37. Why coefficient of variation for consistency? Intermediate

Standard deviation depends on scale; CoV = σ/μ is relative, so drivers with different pace, or different tracks, are compared fairly. It's computed only on clean laps.

Q38. Which data structures did you use and why? Intermediate

TreeMap per driver for ordered laps with O(log n) lookup and easy first/last lap; a precomputed cumulative-time map so a gap is a single subtraction; HashSet for SC laps; ConcurrentHashMap for the thread-safe analyser cache. Times are stored as long milliseconds for exact arithmetic.

Q39. Time complexity of building the analyser? Advanced

Indexing L lap rows is O(L log L) because of the tree inserts. SC detection sorts per-lap times, also roughly O(L log L). Per-request methods are about O(drivers × laps) at worst. With ~1,100 rows it's milliseconds, and it's built once per race and cached.

13.6 Angular & frontend

Q40. What are signals and why use them? Intermediate

Signals are reactive values: signal() for writable state, computed() for memoised derived values, effect() for side effects. Angular knows exactly which views depend on which signals, which enables zoneless change detection and avoids manual subscriptions for state.

Q41. How do components reload data when inputs change? Intermediate

Signal inputs (input.required) are read inside an effect. When raceId or driverId changes, the effect re-runs, fires forkJoin requests, and its onCleanup unsubscribes the previous request, avoiding race conditions with stale responses.

Q42. Standalone components vs NgModules? Basic

Standalone components declare their own imports (for example NgxEchartsDirective, FormsModule), which means less boilerplate, clearer dependencies and better tree-shaking. App-wide providers go in app.config.ts (HttpClient, ECharts).

Q43. How does the frontend talk to the backend in development? Basic

Through the Angular dev-server proxy: /api requests go to localhost:8080. The browser sees one origin, so there's no CORS in development and no hard-coded backend URL in the code.

Q44. How did you make the charts accessible and consistent? Advanced

The colours come from a palette checked for colour blindness, defined as CSS tokens with separate light and dark values, and ECharts reads them at runtime. Selection is capped at 3 series, the only count where all colour pairs stay distinguishable. Each driver keeps their colour while selected. Lines are labelled directly, and there's only one y-axis.

Q45. How did you keep ECharts from bloating the initial load? Intermediate

provideEchartsCore({ echarts: () => import('echarts') }) lazy-loads the library as a separate chunk (~1.1 MB raw, ~310 KB over the network). The initial bundle stays around 260 KB.

13.7 Testing & quality

Q46. How did you test the analytics? Intermediate

With a synthetic race where each lap time follows a formula (base + 0.1 s × tyre age, plus fixed in/out-lap penalties), so every expected value can be worked out by hand: degradation exactly 100 ms/lap, time lost 20.100 s, what-if delta +2.500 s. That proves correctness, not just that nothing changed.

Q47. How did you test code that depends on the LLM? Intermediate

Mock OllamaStatusService and ChatClient (Mockito deep stubs for the fluent API). Tests check the fact-building logic, that the LLM is never called when Ollama is down, and the fallback when the call throws. LLM output itself isn't unit-tested; it's checked with an evaluation set.

Q48. What did the context-load test catch? Basic

The missing entity scan ("Not a managed type"), and later the DDL failure on the unique constraint. Both would have broken the app at runtime even though every unit test passed.

Q49. What would you add to testing next? Intermediate

@WebMvcTest controller tests for status codes and JSON, @DataJpaTest for the projection queries, a WireMock-based test for JolpicaClient paging and merging, Playwright end-to-end tests, and an LLM evaluation suite.

13.8 Behavioural / reflection

Q50. Tell me about a bug you found and how you fixed it. Basic

Chat requests hung for minutes when Ollama wasn't running. The logs showed Spring AI's retry with exponential backoff. I added a 2 s health check before calling the model, limited retries to 2, and added a unit test proving the LLM is skipped when it's down. The fallback now takes about a second.

Q51. A trade-off you made consciously? Intermediate

Deterministic intent parsing instead of tool-calling: less flexible phrasing in exchange for reliability with a small free model. And a linear degradation model: less realistic than a non-linear one, but explainable and stable with few laps. The API returns its assumptions so users know its limits.

Q52. How did you deal with missing data (no tyre compounds)? Basic

I was upfront about it: stints are defined by pit stops, the UI labels it, and the system prompt forbids naming compounds. A future version could add the free OpenF1 API, which has compound data for recent seasons.

Q53. What would you do differently? Intermediate

Set up the LLM evaluation set at the start so prompt changes are measured from day one, use Flyway from the beginning, and stream responses over SSE for better perceived speed.

14. How to run · quick reference

One-time setup (free)

```
winget install Ollama.Ollama        # or download from ollama.com
ollama pull llama3.2                # ~2 GB; optional: ollama pull qwen2.5:7b
```

Start

```
# Terminal 1 – backend (http://localhost:8080)
cd AI
.\mvnw.cmd spring-boot:run

# Terminal 2 – frontend (http://localhost:4200)
cd f1-ui
npm install        # first time
npm start
```

Open `http://localhost:4200/?season=2024&round=1`. Database console: `http://localhost:8080/h2-console` (JDBC URL `jdbc:h2:file:./data/f1db`, user `sa`, empty password).

Tests

```
cd AI;    .\mvnw.cmd test               # 22 JUnit tests
cd f1-ui; npx ng test --watch=false     # Vitest
```

Troubleshooting

"Port 8080 was already in use"	`Get-NetTCPConnection -LocalPort 8080 -State Listen | % { Stop-Process -Id $_.OwningProcess -Force }`
"AI offline" badge	Start Ollama (tray app or `ollama serve`) and run `ollama pull llama3.2`
"Model not ready"	Model still downloading; check with `ollama list`
Import error 502	No internet or Jolpica rate limit; wait and retry
Slow first answer	The model is loading into RAM; later answers are faster

Numbers worth remembering

Race import	~16 HTTP requests, ~17 s, ~1,100 lap rows
AI answer (llama3.2 on CPU)	~27–40 s (was 102 s before optimisation)
Fallback when the LLM is down	~1 s (was minutes)
Thresholds	SC: field median >115%; clean lap: ≤107% of the driver's median; min 3 laps per fit
Tests	22 backend + 2 frontend, all passing

15. Setting up on another machine

Follow this chapter in order on a new computer. Everything is free. Internet is needed only for the first-time downloads (JDK, Node, Ollama, Maven/npm libraries, the AI model) and the first time each race is opened.

15.1 System requirements

Item	Minimum	Recommended (tested)
Operating system	Windows 10/11 64-bit (macOS / Linux also work, see 15.9)	Windows 11 Pro 10.0.26200
RAM	8 GB (llama3.2 needs ~3 GB free while answering)	16 GB+ (32 GB tested; required for `qwen2.5:7b`)
Free disk	~5 GB	10 GB (model 2 GB, Ollama ~1.5 GB, JDK ~0.3 GB, Maven libs ~0.2 GB, node_modules ~0.3 GB)
CPU / GPU	Any 64-bit CPU; no GPU needed	Intel Iris Xe (integrated) tested; an NVIDIA GPU makes answers much faster
Free ports	`8080` (backend), `4200` (frontend), `11434` (Ollama)	
Network (first run)	`repo.maven.apache.org`, `registry.npmjs.org`, `ollama.com` / `registry.ollama.ai`, `api.jolpi.ca`	

15.2 Exact version matrix

These are the versions the project was built and tested with. Use the same major versions; newer patch versions are fine.

Component	Tested version	Required	Where it is defined
JDK	Eclipse Temurin 25.0.2 LTS	**21 or newer** (Java 21 LTS or 25 LTS)	`pom.xml` → `java.version=21`
Maven	3.10.0 (via wrapper 3.3.4)	None to install; `mvnw.cmd` downloads it	`.mvn/wrapper/maven-wrapper.properties`
Spring Boot	4.1.1	fixed	`pom.xml` parent
Spring Framework	7.0.9	managed by Boot	transitive
Spring AI	2.0.1	fixed (2.x is the line for Boot 4)	`pom.xml` → `spring-ai.version`
Hibernate ORM	7.4.5.Final	managed by Boot	transitive
H2 database	2.4.240	managed by Boot	transitive
Tomcat (embedded)	11.0.24	managed by Boot	transitive
Jackson	3.1.5 (`tools.jackson`)	managed by Boot	transitive
JUnit / Mockito	JUnit 6.0.3 · Mockito 5.23.0	managed by Boot	test scope
Node.js	24.18.0	**^22.22.3, ^24.15.0 or ≥26** (Angular 22 requirement)	`@angular/core` engines
npm	12.0.1	8 or newer	`package.json` → `packageManager`
Angular (core, CLI, build)	22.2.3	locked	`package.json` / `package-lock.json`
TypeScript	6.0.3	locked	`package-lock.json`
Apache ECharts / ngx-echarts	6.1.0 / 22.0.0	locked	`package-lock.json`
RxJS · Vitest · jsdom	7.8.2 · 5.0.5 · 30.1.3	locked	`package-lock.json`
Ollama	0.40.4	a recent version	installed app
LLM model	`llama3.2:latest` (3B, ID `a80c4f17acd5` , 2.0 GB)	any Ollama chat model	`spring.ai.ollama.chat.model`
Git (optional)	2.55	any	only if cloning from GitHub

Why exact versions matter: Spring AI 2.x works only with Spring Boot 4.x; Spring AI 1.x will not start on Boot 4. Angular 22 refuses to run on Node 20 or older. `package-lock.json` pins every frontend library, so `npm ci` installs exactly what was tested.

15.3 What to copy (and what to leave out)

Copy	Leave out (rebuilt automatically)

`AI/`: `src/`, `pom.xml`, `mvnw`, `mvnw.cmd`, `.mvn/`
`f1-ui/`: `src/`, `public/`, `package.json`, `package-lock.json`, `angular.json`, `tsconfig*.json`, `proxy.conf.json`
`README.md`, `docs/`
Optional: `AI/data/f1db.mv.db`, which keeps already-downloaded races (~0.3 MB) so they open offline
	`AI/target/` (compiled classes)
	`f1-ui/node_modules/` (~300 MB)
	`f1-ui/dist/`, `f1-ui/.angular/` (build cache)
	`AI/data/*.lock.db` (lock file; only exists while running)

Create a clean zip on the old machine. Stop the backend first so the database file isn't locked:

```
cd C:\Users\<you>\Desktop
robocopy F1 F1-clean /E /XD node_modules target dist .angular /XF *.lock.db
Compress-Archive -Path F1-clean\* -DestinationPath F1-transfer.zip
Remove-Item F1-clean -Recurse -Force
```

The zip is only a few MB. On the new machine, extract it to a path **without spaces or special characters**, e.g. `C:\Projects\F1`.

15.4 Step-by-step installation (Windows)

1. **Install JDK 21+** (Temurin):
   `winget install EclipseAdoptium.Temurin.25.JDK` (or `EclipseAdoptium.Temurin.21.JDK`).
   Close and reopen the terminal, then check: `java -version` should print 21 or higher.
2. **Install Node.js 24 LTS**:
   `winget install OpenJS.NodeJS.LTS`
   Check: `node -v` (v24.x) and `npm -v`.
3. **Install Ollama**:
   `winget install Ollama.Ollama`
   It starts automatically (tray icon) and listens on `http://localhost:11434`. Check: `ollama --version`.
4. **Download the AI model** (2 GB, one time):
   `ollama pull llama3.2`
   Check: `ollama list` shows `llama3.2:latest`.
5. **Extract the project** to e.g. `C:\Projects\F1`.
6. **Build and test the backend** (the first run downloads Maven 3.10.0 and ~170 MB of libraries):
   `cd C:\Projects\F1\AI`
   `.\mvnw.cmd -B test`
   Expected: all tests pass, `Failures: 0, Errors: 0`, then `BUILD SUCCESS`.
7. **Install frontend dependencies** exactly as locked:
   `cd C:\Projects\F1\f1-ui`
   `npm ci`
   `npm ci` installs exactly what `package-lock.json` lists. Use `npm install` only if `npm ci` says the lock file is out of date.
8. **Run the frontend tests**: `npx ng test --watch=false`. Expected: `Tests 2 passed`.
9. **Start the backend** (terminal 1): `cd C:\Projects\F1\AI` then `.\mvnw.cmd spring-boot:run`. Wait for `Started AiApplication`.

10. **Start the frontend** (terminal 2): `cd C:\Projects\F1\f1-ui` then `npm start`. Wait for `Local: http://localhost:4200/`.
11. **Open** `http://localhost:4200/?season=2024&round=1`. The first open downloads the race (~20 s). The chat badge should show ● **llama3.2 (local)**.

15.5 First-run verification checklist

#	Check	Command / URL	Expected
1	Java	`java -version`	21 or higher
2	Node	`node -v`	v22.22.3+, v24.15+ or v26+
3	Ollama + model	`ollama list`	`llama3.2:latest`
4	Ollama API	`http://localhost:11434/api/tags`	JSON listing the model
5	Backend up	`http://localhost:8080/api/races`	`[]` on a fresh machine (or your copied races)
6	AI wiring	`http://localhost:8080/api/ai/status`	`"running":true,"modelAvailable":true`
7	Jolpica reachable	`http://localhost:8080/api/seasons/2024/races`	24 races
8	Database	`http://localhost:8080/h2-console`	Login works with the settings in 15.6
9	Frontend + proxy	`http://localhost:4200/api/races`	Same JSON as #5 (proves the proxy works)
10	End to end	Open a race, ask "Summarize the race strategy"	An answer within ~30–90 s, with "Data used"

15.6 Configuration reference

All backend settings live in `AI/src/main/resources/application.properties`. Any property can also be overridden without editing the file, for example `.\mvnw.cmd spring-boot:run "-Dspring-boot.run.arguments=--server.port=9090"`.

Property	Value	Change it when…
`server.port`	`8080`	port 8080 is taken (also update `proxy.conf.json`)
`spring.datasource.url`	`jdbc:h2:file:./data/f1db;AUTO_SERVER=TRUE`	you want the DB elsewhere. The path is relative to the folder you start from, i.e. `AI/`
`spring.datasource.username` / `password`	`sa` / *(empty)*	never needed locally
`spring.jpa.hibernate.ddl-auto`	`update`	no change needed; tables are created automatically on first start
`spring.h2.console.enabled` / `path`	`true` / `/h2-console`	set to false if you don't want the console
`spring.ai.ollama.base-url`	`http://localhost:11434`	Ollama runs on another PC (e.g. a GPU machine)
`spring.ai.ollama.chat.model`	`llama3.2`	using another model, e.g. `qwen2.5:7b` (run `ollama pull` first)
`spring.ai.ollama.chat.temperature`	`0.2`	keep it low for factual answers
`spring.ai.ollama.chat.num-ctx`	`4096`	raise to 8192 only for very long questions (slower)
`spring.ai.retry.max-attempts`	`2`	keep it low so the app falls back fast when Ollama is down
`f1.jolpica.base-url`	`https://api.jolpi.ca/ergast/f1`	the API moves
`f1.jolpica.request-delay-ms`	`350`	you hit rate limits (raise to 600)
`f1.cors.allowed-origins`	`http://localhost:4200`	the frontend is served from another origin

Frontend: `f1-ui/proxy.conf.json` forwards `/api` to `http://localhost:8080`. Change the target if the backend port changes. To change the dev-server port, run `npm start -- --port 4300`.

H2 console login: JDBC URL `jdbc:h2:file:./data/f1db` · user `sa` · empty password. Type the URL exactly; the default `jdbc:h2:~/test` opens a different, empty database.

15.7 Moving the data and the model (optional, saves downloads)

- **Imported races:** copy `AI/data/f1db.mv.db` with the backend stopped. Those races then open instantly and offline. The pinned Spring Boot version brings the same H2 version (2.4.x), which can read the file.

- **AI model without re-downloading 2 GB:** copy `%USERPROFILE%\.ollama\models` from the old PC to the same path on the new PC (with Ollama closed on both), then check with `ollama list`.
- **Maven libraries offline:** copy `%USERPROFILE%\.m2` (~170 MB; it includes the cached Maven distribution under `wrapper`) and run `.\mvnw.cmd -o spring-boot:run`. The `-o` flag means offline.
- **npm offline:** copying `node_modules` between Windows machines usually works, but `npm ci` is safer because some packages contain native binaries.

15.8 Troubleshooting on a new machine

Symptom	Cause	Fix
`'java' is not recognized` or `JAVA_HOME not found`	JDK not on PATH	Reopen the terminal after installing; or set `JAVA_HOME` to the JDK folder (e.g. `C:\Program Files\Eclipse Adoptium\jdk-25…`) and add `%JAVA_HOME%\bin` to PATH
`release version 21 not supported`	JDK 17 or older	Install JDK 21+ and make it the first `java` on PATH
`npm.ps1 cannot be loaded … running scripts is disabled`	PowerShell execution policy	`Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`, or call `npm.cmd` instead of `npm`
`Node.js version vX is not supported by Angular CLI`	Node too old	Install Node 24 LTS (or with nvm-windows: `nvm install 24` then `nvm use 24`)
`Port 8080 was already in use`	Another app or an old backend	`Get-NetTCPConnection -LocalPort 8080 -State Listen | % { Stop-Process -Id $_.OwningProcess -Force }` or change `server.port`
UI shows "Backend not reachable"	Backend not started, or the proxy points to the wrong port	Start the backend; check `proxy.conf.json`
Chat badge "○ AI offline"	Ollama not running	Start the Ollama app from the Start menu, or run `ollama serve`
"Model not ready (Run: ollama pull llama3.2)"	Model missing or still downloading	`ollama pull llama3.2`; check `ollama list`
Maven / npm / Ollama downloads fail on a company network	Corporate proxy or SSL inspection	Maven: add a `<proxy>` to `%USERPROFILE%\.m2\settings.xml`. npm: `npm config set proxy http://host:port` and `https-proxy`. Ollama: set the environment variable `HTTPS_PROXY` and restart Ollama
Race import returns 502	No internet or Jolpica rate limit (500 requests/hour)	Open `https://api.jolpi.ca/ergast/f1/2024.json` in a browser; wait and retry
`Database may be already in use`	A stale lock after a crash	Stop all Java processes, then delete `AI/data/f1db.lock.db`
Answers are very slow (>2 min)	Low RAM, a large model, or the first load	Close heavy apps; keep `llama3.2`; the first answer after a start is always slower
Windows Firewall prompt for Java/Node/Ollama	Local servers opening ports	Allow on **Private** networks only (everything uses localhost)

15.9 macOS / Linux equivalents

```
# macOS (Homebrew)                          # Ubuntu / Debian
brew install --cask temurin                  sudo apt install openjdk-21-jdk
brew install node@24                         nvm install 24        # Node via nvm
brew install ollama && ollama serve         curl -fsSL https://ollama.com/install.sh | sh
ollama pull llama3.2                         ollama pull llama3.2

# both
cd AI && chmod +x mvnw && ./mvnw spring-boot:run
cd f1-ui && npm ci && npm start
```

15.10 One-page quick start (print this)

```
1. winget install EclipseAdoptium.Temurin.25.JDK
   winget install OpenJS.NodeJS.LTS
   winget install Ollama.Ollama
2. (reopen the terminal)   ollama pull llama3.2
3. cd C:\Projects\F1\AI      ;  .\mvnw.cmd spring-boot:run      # terminal 1
4. cd C:\Projects\F1\f1-ui   ;  npm ci  ;  npm start            # terminal 2
5. open http://localhost:4200/?season=2024&round=1
   DB console: http://localhost:8080/h2-console  (jdbc:h2:file:./data/f1db · sa · no password)
```